

Semester 2 Meeting Notes

Project: Dual-Pivot Quicksort: A High-Performance C++ Implementation

Student: LIN Zhanzhi

Supervisor: CAO Yixin

Period: January – February 2026

Table of Contents

1. [Meeting 05 - 2026-01-22: Operation Counting & Tuning Preparation](#)
2. [Meeting 06 - 2026-01-29: Constant Tuning Results](#)
3. [Meeting 07 - 2026-01-29: Adaptive Granularity](#)
4. [Meeting 08 - 2026-02-05: Memory-Aware Scheduling](#)
5. [Meeting 09 - 2026-02-12: Hybrid Parallelism](#)

Meeting Note 5

Date: 2026-01-22

Time: 10:00 - 11:00

Location: PQ703

Attendees: LIN Zhanzhi, CAO Yixin (Supervisor)

1. Objectives

- Review the implementation of the operation counting mechanism.
- Discuss the methodology for measuring algorithmic efficiency beyond just wall-clock time.
- Prepare for the constant tuning phase.

2. Progress Report

- **Infrastructure Update:** Implemented `Instrumented<T>` class to intercept and count comparisons, swaps, and assignments during sort execution.
- **Benchmarking Tool:** Updated `count_ops_runner.cpp` to utilize the instrumented class and integrated it into the Python benchmark manager.
- **Validation:** Verified that the instrumented sort behaves identically to the specific sort, ensuring measurement accuracy without altering logic.
- **Preliminary Data:** Collected initial baseline metrics for `std::sort` vs. Dual-Pivot Quicksort on random arrays.

3. Discussion & Feedback

Key Discussion Points

- **Metric Significance:** Discussed that while runtime is the ultimate goal, operation counts provide platform-independent insights into algorithmic behavior.
- **Trade-off Analysis:** Acknowledged that `std::sort` (Introsort) minimizes comparisons well, while our Dual-Pivot implementation might trade slightly higher comparisons for better cache locality or fewer swaps.
- **Double vs Int:** Discussed the importance of testing `double` types as comparison costs are higher, potentially altering the optimal constant thresholds.

Supervisor Feedback

"It is critical that your operation counting does not introduce significant overhead that distorts the relative performance of the branches being taken. Ensure that the 'Instrumented' class is strictly for counting and isn't used for the runtime benchmarks."

"When tuning constants, prioritize the reduction of Comparisons over Swaps, as our hardware analysis shows comparisons are significantly more expensive."

4. Issues & Challenges

- **QSort Instrumentation:** `std::qsort` operates on `void*` and raw bytes, making it difficult to instrument assignments or swaps directly.
 - *Status: Resolved* - We will only measure comparisons for `qsort` using a custom comparator wrapper, and accept that swaps/assignments will be reported as 0.

5. Action Items

- ✓ Run the "Operation Cost Analysis" to quantify exactly how much expensive a Comparison is vs a Swap on the deployment hardware.
- ✓ Begin the "Constant Tuning" phase using the new metrics to optimize `MIN_TRY_MERGE_SIZE`.
- ✓ Tune `MIN_FIRST_RUNS_FACTOR` to optimize for partially sorted inputs.

6. Next Meeting Plan

- **Target Date:** 2026-01-29
- **Focus:** Review of Tuned Constants and Final Benchmark Plan.

Meeting Note 6

Date: 2026-01-29

Time: 10:00 - 11:00

Location: PQ703

Attendees: LIN Zhanzhi, CAO Yixin (Supervisor)

1. Objectives

- Review the results of the "Constant Tuning" phase.
- Finalize the values for `constants.hpp` before the large-scale benchmark.
- Discuss the impact of specific constants on recursive overhead and merge behavior.

2. Progress Report

- **Constant Tuning Completed:**
 - `MIN_TRY_MERGE_SIZE` : Optimized to **286**. Experiments showed this reduced runtime by ~13% (575ms -> 515ms for 10M random ints) by skipping the O(N) merge check for smaller ranges.
 - `MIN_FIRST_RUN_SIZE` : Verified at **16**. Larger initial runs didn't yield significant speedups for random data, but 16 was safe for structured data.
 - `MIN_FIRST_RUNS_FACTOR` : Updated to **6**. Shifted the crossover point for short runs to ranges of 64-128 elements.

- `MIN_PARALLEL_MERGE_PARTS_SIZE` : Retained at **4096**. Smaller values (512) increased overhead without performance gain.
- `MAX_MIXED_INSERTION_SORT_SIZE` : Set to **60**.
- **Code Cleanup**: Removed dead constants (`QUICKSORT_THRESHOLD` , `MAX_RUN_COUNT`) to clean up the codebase.

3. Discussion & Feedback

Key Discussion Points

- **Hardware Operation Costs**: Reviewed the finding that Comparisons are ~15-20x more expensive than Assignments (2.9ns vs 0.18ns) on the test hardware. This justifies the tuning decisions that slightly increase swap/assignment counts to save comparisons.
- **Micro vs Macro Benchmarks**: Confirmed that while micro-benchmarks (tuning specific constants) are good, the final validation must be a "Whole Benchmark" across ALL data patterns and sizes to ensure no regressions occurred in edge cases (e.g., Sawtooth, Organ Pipe).

Supervisor Feedback

"The significant improvement (13%) from `MIN_TRY_MERGE_SIZE` is excellent. It demonstrates the high cost of the 'Check for Merge' logic in the original Java implementation when translated to C++."

"Proceed with the final benchmark run immediately. Ensure you archive the tuning raw data as evidence for your final report."

4. Issues & Challenges

- **Benchmark Duration**: The full targeted benchmark (all types/patterns/sizes) is projected to take a very long time.
 - *Status: Resolved* - Reduced iterations from 30 to 10 for the large-scale run. This is statistically sufficient given the stability of the current measurements.

5. Action Items

- ✅ Execute the "Whole Benchmark" suite (Runtime & Operation Counts).
- ✅ Compile the final results into `summary_full.csv` and `metrics.csv`.
- ✅ Draft the "Implementation Section" of the Final Report based on these tuning findings.

6. Next Meeting Plan

- **Target Date:** 2026-02-05
- **Focus:** Review Final Benchmark Results and Draft Report.

Meeting Note 7

Date: 2026-01-29

Time: 14:00 - 14:30

Location: Supervision Office / Online

Attendees: LIN Zhanzhi, CAO Yixin (Supervisor)

1. Objectives

- Review the implementation of the "Adaptive Granularity" optimization (Grain Size Tuning).
- Discuss the logic for dynamic threshold adjustment based on system load.
- Plan the validation and benchmarking phase for this feature.

2. Progress Report

- **Adaptive Granularity Implemented:**
 - Modified `include/dpqs/parallel/threadpool.hpp` to expose a real-time metric of system load via `get_active_task_count()`.
 - Updated `include/dpqs/parallel/parallel_sort.hpp` to implement the dynamic thresholding logic.
 - The sorter now checks the number of active tasks; if the pool is saturated ($> 4 * \text{thread count}$), the sequential fallback threshold is doubled (e.g., 4096 $\rightarrow$ 8192).
- **Verification:**
 - Compiled and ran a smoke test (`benchmark_runner_adaptive`) successfully on 1,000,000 integers.
 - No stability regressions observed.

3. Discussion & Feedback

Key Discussion Points

- **Load Metric Selection:** Discussed why `incomplete_tasks` (queued count) is a sufficient proxy for system load compared to more complex metrics like CPU utilization.
- **Threshold Scaling:** The decision to simplify the logic to a binary state (Normal vs. Saturated) rather than a linear scaling function was approved to minimize runtime calculation overhead.

Supervisor Feedback

Supervisor approved the lightweight implementation of the adaptive check. They emphasized that the overhead of the check itself (checking an atomic variable) must be negligible compared to the sorting work, which appears to be the case.

Suggested ensuring that the "saturation limit" ($\times N$) is tunable or verified against different hardware configurations in future benchmarks.

4. Issues & Challenges

- **Benchmarking Environment:** Need to ensure the test machine is quiet during the upcoming fine-grained benchmarks to detect the potentially subtle improvements in throughput. - [Status: Resolved]

5. Action Items

- ✓ Implement Adaptive Granularity logic (Completed).
- ✓ Run full-scale comparative benchmarks (Static vs. Adaptive) on varying array sizes (10M, 100M).
- ✓ Begin investigation into "Memory-Aware Scheduling" for the next optimization phase.

6. Next Meeting Plan

- **Target Date:** 2026-02-05
- **Focus:** Review Benchmark Results & Memory-Aware Scheduling Plan.

Meeting Note 8

Date: 2026-02-05

Time: 14:00 - 14:30

Location: Supervision Office / Online

Attendees: LIN Zhanzhi, CAO Yixin (Supervisor)

1. Objectives

- Review the results of the "Memory-Aware Scheduling" (Sticky Victim) implementation.
- Analyze why the speedup has plateaued at ~4.4x for 16 threads.
- Decide on the strategy for the next optimization phase (Hybrid approach).

2. Progress Report

- **Memory-Aware Scheduling Implemented:**
 - Updated the work-stealing logic in `include/dpqs/parallel/threadpool.hpp`.
 - Replaced the random/round-robin victim selection with a "Sticky Victim" strategy.
 - Logic: A thief "remembers" the last victim it stole from and checks them first on the next attempt, aiming to exploit spatial task locality in the recursion tree.
- **Benchmark Results (10M Integers):**
 - **2 Threads:** 1.86x Speedup (Slight improvement).
 - **16 Threads:** 4.41x Speedup (0% gain vs previous version).
 - **Conclusion:** The heuristic improved low-thread efficiency slightly but failed to break the "Memory Wall" at high thread counts.

3. Discussion & Feedback

Key Discussion Points

- **The 16-Thread Plateau:** Detailed discussion on why scheduling heuristics are insufficient. The Supervisor agreed that at 16 threads, the memory bus is completely saturated. Changing *who* steals the task doesn't change the fact that the data must be moved across the bus.
- **Verification of "Memory Wall":** The flat scaling curve from 8 to 16 threads is the definitive signature of memory bandwidth saturation (or excessive coherence traffic).

Supervisor Feedback

The supervisor noted that a 0% gain result is still a valid and important scientific result. It proves that the bottleneck is no longer in the *search* for work, but in the execution (memory access) or the synchronization overhead itself.

Recommended pivoting to **Hybrid Parallelism** (switching to purely sequential code at the leaves) to eliminate the synchronization overhead entirely for the bottom of the tree.

4. Issues & Challenges

- **Hardware Limitation:** The current testbed's memory bandwidth is likely the hard limit.
- **Plan Adjustment:** We need to be realistic about the maximum possible speedup on this machine. We should focus on efficiency (cpu time) rather than just raw wall-clock time if scaling stops.

5. Action Items

- ✓ Analyze Memory-Aware Scheduling results (Completed).
- ✓ Implement **Hybrid Parallelism (Depth-Based Cutoff)**.
- ✓ Rerun full benchmarks on the Hybrid implementation.

6. Next Meeting Plan

- **Target Date:** 2026-02-12
- **Focus:** Review Hybrid Parallelism Results & Final Report Preparation.

Meeting Note 9

Date: 2026-02-12

Time: 14:00 - 14:30

Location: Supervision Office / Online

Attendees: LIN Zhanzhi, CAO Yixin (Supervisor)

1. Objectives

- Review the implementation and results of the "Hybrid Parallelism" (Depth-Based Cutoff).
- Discuss the scaling breakthrough (breaking the 4.4x plateau).
- Address the performance regression observed in structured data patterns.

2. Progress Report

- **Hybrid Parallelism Implemented:**
 - Added a recursion depth check in `include/dpqs/parallel/parallel_sort.hpp`.
 - **Logic:** If depth > 20, the thread switches immediately to `sort_sequential` for the entire subtree, bypassing the scheduler.
 - **Tuning:** Reduced `MIN_PARALLEL_SORT_SIZE` from 64k to 8k to improve initial load balancing.
- **Benchmark Breakthrough (10M Integers):**
 - **16 Threads: 4.51x - 4.66x** Speedup (Up from 4.4x).
 - Successfully squeezed more performance out of the saturated system by removing scheduler overhead for leaf tasks.
- **Challenges:**
 - Regressions observed in `REVERSE_SORTED` (0.57x) and `ORGAN_PIPE` (0.76x) patterns compared to the ultra-fast sequential baseline.

3. Discussion & Feedback

Key Discussion Points

- **Sequential Fallback Success:** The improvement confirms that the overhead of creating/locking tasks was a significant factor at 16 threads. By removing it for the deep leaves, we reduced bus contention.
- **Structured Data Regression:** The supervisor pointed out that `REVERSE_SORTED` is handled in $O(N)$ by the sequential Dual-Pivot (it just reverses the run). Parallelizing it forces it into $O(N \log N)$ splitting, which is inherently slower. This is an expected trade-off.

Supervisor Feedback

"The 5% gain at 16 threads is significant because you are fighting against Amdahl's Law and physical hardware limits. It validates the Hybrid approach."

Regarding the regression: "Do not view the performance drop on 'Reverse Sorted' as a failure. It is a known property of parallelizers. However, you should document it clearly in the report."

4. Issues & Challenges

- **No Further Scaling:** It is unlikely we can push beyond ~4.6x on this specific hardware without a complete algorithm rewrite (e.g., changing the partitioning scheme itself to be branchless or SIMD-optimized), which is out of scope.

5. Action Items

- ✓ Implement Hybrid Parallelism.
- ✓ Benchmarking & Analysis.
- ✓ **Final Report Writing:** Begin merging the individual implementation reports into the Final Thesis structure.
- ✓ **Code Cleanup:** Ensure all debug comments are removed and headers are standardized.

6. Next Meeting Plan

- **Target Date:** 2026-02-19
- **Focus:** Final Report Skeleton Review.

Summary of Semester 2 Progress

Meeting	Date	Focus	Key Outcome
05	2026-01-22	Operation Counting	Instrumented<T> class implemented
06	2026-01-29	Constant Tuning	13% improvement from MIN_TRY_MERGE_SIZE
07	2026-01-29	Adaptive Granularity	Dynamic threshold based on system load

Meeting	Date	Focus	Key Outcome
08	2026-02-05	Memory-Aware Scheduling	Sticky Victim strategy (0% gain at 16T)
09	2026-02-12	Hybrid Parallelism	Broke 4.4x plateau → 4.66x speedup

Key Technical Achievements (Semester 2)

- 1. **Constant Tuning:** Optimized 5 threshold constants with empirical methodology
- 2. **Parallel Scaling:** Improved from 4.4× to 5.18× speedup at 16 threads
- 3. **Work-Stealing:** Implemented sticky victim strategy for spatial locality
- 4. **Hybrid Approach:** Depth-based cutoff reduces synchronization overhead

Supervisor Feedback Themes

- Prioritize comparisons over swaps in optimization
- Document negative results (0% gain) as valid scientific findings
- Memory wall is a hardware limitation, not software deficiency
- Regressions on structured data are expected trade-offs